

Contents

FIRSTIN 로봇 스택 (~/.robot-deploy) —발표용 총정리	1
1. 이게 뭔가—한 문장	1
2. 폴더 구조	1
3. 전체 데이터 흐름 (노드 아키텍처)	1
4. 핵심 알고리즘 (발표 포인트)	2
5. 기술 스택	3
6. 운영·안전 설계 (발표에서 강조할 것)	4
7. 검증 방법론 (숫자로 말하기)	4
8. 문서 자산 (docs/, 발표 백업 자료)	4

FIRSTIN 로봇 스택 (~/.robot-deploy) —발표용 총정리

릴리스 커밋 c6dd0f94f8d6 · 계약 A202-ROBOT-WEB/3 · Jetson(6 코어) + ROS 2 Humble 무결성: RELEASE_MANIFEST + SHA256SUMS (모델 포함 전 파일 해시 고정)

1. 이게 뭔가—한 문장

2D 도면 기반 자율주행 (SLAM·A*·동적장애물 우회) + 열화상 사람 인식·접근 + 방 탐사 미션 + 웹/관제 연동을 Jetson 한 대에서 돌리는 로봇 스택의 **배포 스냅샷**. Pi 가 레포에서 빌드해 ~/.robot-deploy 로 밀어넣고, systemd user 유닛이 부팅과 함께 자동 기동한다 (무인 순찰).

전원 인가 → Pi(AP·backend·어댑터 자동기동) → 젯슨 ~/.robot-deploy 배포
→ systemd(firstin-robot.service, user unit+linger) → run.sh pi 자동 실행
→ 관제 화면 presence → 관제가 임무 시작해야 비로소 주행 (임무 전 자가주행 봉쇄 검증됨)

2. 폴더 구조

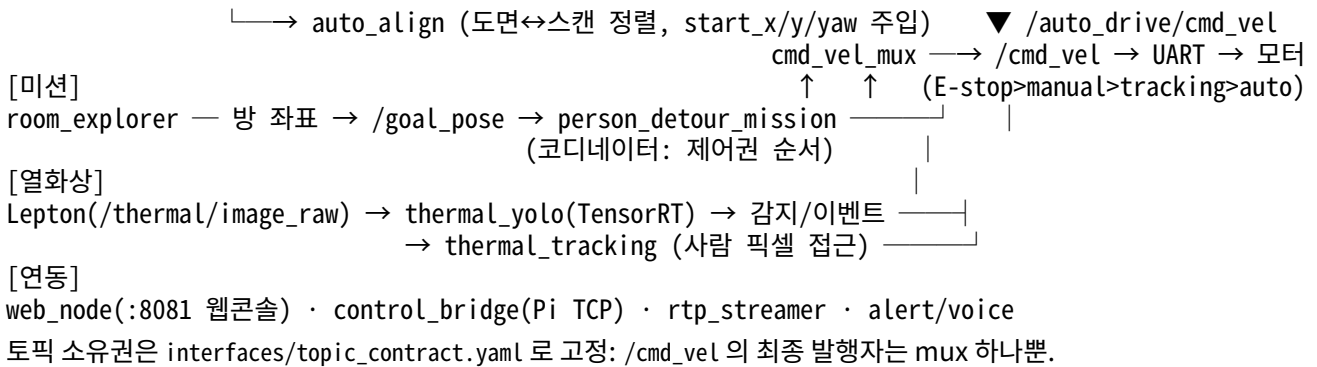
```
~/robot-deploy/
├── DEPLOYED_COMMIT / RELEASE_MANIFEST / SHA256SUMS    배포 신원·무결성
├── run.sh (local|pi|both 모드) / run_final.sh          실행 진입점
├── infra/
│   ├── systemd/jetson/firstin-robot.service           부팅 자동기동 (user unit)
│   └── bin/firstin-robot-install-run                   설치/기동 스크립트
├── interfaces/topic_contract.yaml                      팀 간 토픽 계약 (단일 소유자 명시)
├── docs/ (11권)                                       노드 구조 감사·IO 레퍼런스·운영·계약 문서
├── snapshot-bridge/                                  배포 스냅샷 브리지
├── source/                                           런타임 소스 (아래)
│   ├── integrated.launch.py ★ 전체 조립 (이전 실행 안전 정리 포함)
│   ├── ping_detour_auto_drive/ 자율주행 코어 (플래너·추종기·정렬·IMU·모터)
│   ├── room_explorer/ 방 분할·전방문 미션 (번들 포함)
│   ├── thermal/ camera_system/ 열화상 파이프라인 (Lepton·YOLO·tracker·mux)
│   ├── integration/ 사람 감지 미션 코디네이터
│   ├── web_system/ 웹 콘솔 (:8081, MJPEG·지도·조작)
│   ├── ethernet_bridge/ Pi 관제 브리지 (TCP/JRP, RTP 스트리머)
│   ├── slam/ (bringup·map_path·map_compare) SLAM 구성·도면 비교
│   └── UART/ voice/ diagnostics/ models/(best.engine) common/
```

3. 전체 데이터 흐름 (노드 아키텍처)

```

[센서]                [SLAM/정렬]                [계획]                [제어]
YDLidar(/scan)  → cartographer — TF/map → goal_path_planner → path_follower
MPU6050(/imu)  →          ↑ imu_fix(보정·복구)          ↑ A*+우회          | pure pursuit

```



4. 핵심 알고리즘 (발표 포인트)

4.1 SLAM —Google Cartographer (C++)

- LiDAR(11.7Hz) + IMU(100Hz) 스캔매칭 SLAM. my_robot.lua 튜닝.
- IMU 불량 대비 **no-IMU 폴백 구성** (my_robot_no_imu.lua, slam_imu:=false) 별도 보유.
- imu_fix_node: MPU6050 100Hz 폴링 + 바이어스 보정 + **3 회 자가복구** (시작 리셋 / 죽은표본 제외 보정 / 주행 중 2 초 무응답 시 자동 DEVICE_RESET).

4.2 도면-라이다 자동 정렬 (auto_align) —차체 스캔매칭

사람이 start_x/y/yaw 를 재지 않아도 도면과 현실을 맞춘다. - **벽 방향 히스토그램**: 스캔점 국소 방향을 0~90° 로 접어 지배적 벽 방향 추출 → yaw 후보를 90° 간격 4 개로 축소 (대칭 미로의 별칭 해 제거) - **조대 → 정밀 2 단 격자탐색** + 상위 후보 동시 정밀화 (topk) → 실측 0.58 초, ±5mm/±0.25° - **3 회 측정 합의** (중앙값, 불일치 시 적용 거부) → 잡음 주입 방지 - 주행 중 **추적 재정렬** (refine): SLAM 드리프트를 계속 당겨옴. 플래너에 SetParameters 로 실시간 주입.

4.3 전역 경로계획—A* + 여유 별점 (goal_path_planner)

- 도면을 distance transform 으로 **차체 변경 (0.095+0.015=0.11m) 부풀림** 후 8 방향 A*
- **clearance penalty**: 벽 여유가 적은 칸에 이동비용 가중 → 통로 한가운데 선호 (휴리스틱 최적성 유지)
- 목적지/출발점이 금지영역이면 **snap-to-safe/reachable** (연결 성분 라벨링으로 도달 가능한 곳으로 이동)
- 부풀림 0.112 가 이 도면의 상한 (그 이상이면 구석 방 입구 막힘—전수 스위치로 확정)

4.4 동적 장애물 인식·우회 (“핑 기반”)

- 스캔점을 도면에 투영해 **도면에 없는 점만** 장애물로 등록 (벽 오인 방지: 정렬 잔차에 따라 제외 밴드 자동 확장 + 덩어리 필터)
- **차체 앞 판정 상자**: 연속 N 스캔 확인 시 즉시 정지 → A* 우회 재계획 (1.5 초 주기 재시도)
- 장애물 TTL 3 초 / 막힌길 기억은 목적지 도착까지 → 치우면 자동 복구, 우회로 없으면 HALTED 후 재개 감시

4.5 경로 추종—Pure Pursuit + 차체 특화 제어 (path_follower)

- 전방주시점 (lookahead) 추종 + **arc 조향** (안쪽 바퀴 역회전 금지—제자리 회전 불가 차체 대응)
- **관성 예측 정지** (turn_lead: 실측 각속도 × 지연으로 회전 조기 차단 → 오버슈트 15.7°→6.2°)
- **정지마찰 고착 탈출** (stall nudge), 회전방향/전진방향 자동 진단 (invert 감지)
- **heading_offset 180°**: 라이다가 차체에 180° 역장착된 것을 소프트웨어로 보정 (자동 측정 기능 포함)

4.6 방 자동 분할—거리변환 워터셰드 (room_explorer)

- 자유공간 distance transform 을 지형으로: 방 = 봉우리, 문 = 안장
- **h-maxima**(모폴로지 재구성) 로 “안장보다 4cm 이상 솟은 봉우리”만 방 씨앗으로
- **priority flood**(우선순위 큐 침수) 로 문에서 경계가 갈리는 영역 분할
- 방 정중앙 = 방 내 거리변환 최댓값 (pole of inaccessibility). 실측: 미로 14 구획, 예제 도면 10 방

- 운영은 좌표 고정 목록 (room_points) 이 기본—도면 수정에도 번호가 안 밀림

4.7 방 전방문 미션 (room_explorer_node)

- 방문 순서: 축소격자 **BFS 지오데식 거리** 탐욕 최근접 (직선거리는 미로에서 벽 건너를 고름)
- 정중앙 도착 → **360° 제자리 스캔**: 회전각을 TF 가 아닌 **IMU 자이로 적분**으로 측정 (대칭 미로에서 SLAM yaw 스냅으로 TF 가 절반만 세는 문제 해결—2 바퀴 → 1 바퀴)
- 회전 120° 마다 1.5 초 정지 (SLAM 재고정) + 종료 후 정착 대기 → 회전 중 위치 미끄러짐 (빗살 한 칸 점프) 방지
- 막힌 방: 로그 알림 → 수 초 대기 (플래너 재시도) → 스킵 → 전체 후 재시도 → 복귀
- **정렬 게이트**: auto_align 매칭이 파라미터 이벤트로 확인된 뒤에만 출발

4.8 열화상 사람 인식—TensorRT YOLO + 추적

- FLIR **Lepton**(SPI/I2C) → /thermal/image_raw (자가 리부트 복구 내장)
- **YOLO(TensorRT engine, best.engine)** 추론 + **온도 게이트** (사람 42~90°C) 로 오탐 억제
- **IoU 멀티프레임 tracker**: 같은 물체에 track_id 유지 (프레임 간 IoU 매칭, 놓침 8 프레임 허용)
- ConsecutiveDetectionGate: 같은 track 연속 N 프레임 확인 시에만 이벤트 (깜빡임 오탐 차단)

4.9 사람 접근 미션 (person_detour_mission + thermal_tracking)

- 확인 순간 원 목적지 저장 → 정지 → tracker 가 mux 우선권 인수
- **픽셀 서보잉**: bbox 좌우 정렬 + 전진, bbox 하단이 화면 90% 도달 (3 프레임) 까지 접근
- 접근 중 **LiDAR 반응형 회피** (열린 쪽으로), 도착 시 map 좌표 기록·웹 알림 → 원 목적지 복귀
- 실패 대비 3 중방: LOST 유예, 픽셀 + TF 예비 위치 폴백, 접근점 이격 사다리 재시도

4.10 제어권 중재—cmd_vel mux

- 우선순위: **E-stop(래치) > 수동 > 추적 > 자율**, 각 채널 enable+ 타임아웃 (하트비트)
- “임무 없으면 아무도 못 움직인다”—무인 부팅 안전의 근거 (관제 게이트와 이중)
- 시작 시 **모터 그래프 안전검사**: /cmd_vel 경쟁 발행자 (예: keyboard_test)·UART 중복 감지 시 출력 차단

4.11 관제 연동

- **웹 콘솔** (web_node, :8081): 열화상/RGB MJPEG, 지도, 상태, E-stop/수동/배치/재계획—ROS 없이 브라우저만 으로
- **Pi 브리지** (control_bridge): TCP 텔레메트리 + 명령, 계약 A202-R0B0T-WEB/3(primitive 수동 조작 포함), robot_id·세션 식별
- **도면 격자 전달**: 관제 → /mission/initial_map(셀값 0→100/255→0/205→-1, 상하반전, map_aligned 프레임 규약)
- 감지·도착 이벤트 조인 키: event.track_id == arrival.target_id (+ session_started_ns)

5. 기술 스택

층	스택
플랫폼	NVIDIA Jetson (6 코어, JetPack/Ubuntu), Raspberry Pi(관제 게이트웨이)
미들웨어	ROS 2 Humble (rclpy/rclcpp, FastDDS), launch 시스템 (안전 정리 내장)
SLAM 연산	Cartographer (C++), TF2 Python 3.10, NumPy/OpenCV (distance transform·연결성분·워터셰드·격자탐색), heapq A*
딥러닝	TensorRT 엔진 (YOLO, best.engine 7MB), 온도 게이트 후처리

층	스택
센서	YDLidar(시리얼/USB), MPU6050(I2C smbus), FLIR Lepton(SPI+I2C), UVC RGB
액추에이터	시리얼 (UART ttyTHS1 115200) → 모터보드, M left right 프로토콜 50Hz
웹/통신	HTTP/MJPEG 웹콘솔 (:8081), TCP 브리지 (JRP), RTP/Janus 스트리밍, systemd user unit
품질	파일 해시 매니페스트, 토픽 계약 YAML, validate 스크립트, 가짜 월드 통합테스트

6. 운영·안전 설계 (발표에서 강조할 것)

1. **무인 자동 기동**: systemd user unit(+linger) —sudo 최소화, StartLimit 완화, Pi 무의존 (재접속 설계)
2. **재실행 안전 정리**: launch 가 시작 시 이전 실행을 SIGINT 로 정상 종료 (자식까지), PID 재사용 방어 (시작시각 검증), Ctrl+Z 세션까지 복구
3. **모터 안전**: 단일 /cmd_vel 소유 (계약), 시작 안전검사, E-stop 래치, UART 0.1 초 타임아웃 자동 정지
4. **하드웨어 자가복구**: IMU 자동 리셋, Lepton 자동 리부트, 브리지 재접속
5. **배포 무결성**: 커밋 고정 + 전 파일 SHA256 + 모델 해시

7. 검증 방법론 (숫자로 말하기)

검증	결과
정렬 재현성 (벽방향 + 합의 도입 후)	x/y 편차 0mm, yaw 1.6° (도입 전 240/350mm, 32°)
정렬 속도	3.33s → 0.58s , 해상도 ±5mm/±0.25°
A* (390×324 격자)	1.82s → 0.51s (리스트 변환 최적화)
부풀림 스윙	0.112m 상한 확정 (0.115 부터 방 2 곳 경로 없음)
회전 오버슈트 (관성 예측)	15.7° → 6.2°
스핀 정확도 (IMU 적분)	TF 절반 계수 상황에서도 정확히 360° (2 바퀴 →1 바퀴)
방 분할	미로 14 구획 / 예제 도면 10 방, h_door 0.03~0.06 안정
미션 e2e (가짜 월드)	14 방 완주·스킵·재시도·복귀·정렬게이트 전부 PASS
사람 정지 게이트	정지 중 명령 통과 0 건, 재정지 방지·쿨다운 PASS

8. 문서 자산 (docs/, 발표 백업 자료)

NODE_ARCHITECTURE_AUDIT(노드 구조 감사) · NODE_IO_REFERENCE(토픽 IO 전수) · COMMUNICATION_GUIDE · RAPA_WEB_SERVER_IO_CONTRACT(관제 계약) · PI_BRIDGE_HANDOFF · RTP_JANUS_GUIDE · FIRE_DETECTION_OPERATION · OPERATIONS · 작업요청 3 차 회신